

Performance Evaluation of Intrusion Detection Streaming Transactions Using Apache Kafka and Spark Streaming

May Thet Tun, Dim En Nyaung, Myat Pwint Phyu

University of Information Technology, Yangon, Myanmar

maythettun@uit.edu.mm, dimennyaung@uit.edu.mm, myatpwintphyu@uit.edu.mm

Abstract

In the information era, the size of network traffic is complex because of massive Internet-based services and rapid amounts of data. The more network traffic has enhanced, the more cyberattacks have dramatically increased. Therefore, cybersecurity intrusion detection has been a challenge in the current research area in recent years. The Intrusion detection system requires high-level protection and detects modern and complex attacks with more accuracy. Nowadays, big data analytics is the main key to solve marketing, security and privacy in an extremely competitive financial market and government. If a huge amount of stream data flows within a short period time, it is difficult to analyze real-time decision making. Performance analysis is extremely important for administrators and developers to avoid bottlenecks. The paper aims to reduce time-consuming by using Apache Kafka and Spark Streaming. Experiments on the UNSWNB-15 dataset indicate that the integration of Apache Kafka and Spark Streaming can perform better in terms of processing time and fault-tolerance on the huge amount of data. According to the results, the fault tolerance can be provided by the multiple brokers of Kafka and parallel recovery of Spark Streaming. And then, the multiple partitions of Apache Kafka increase the processing time in the integration of Apache Kafka and Spark Streaming.

Key Words- Intrusion Detection, Big Data, Apache Kafka, Apache Spark Streaming

1. Introduction

In 2020, nearly 50 billion devices are going to connect the Internet as a result of the extensive area of communication and advanced technologies [15]. The protection of secure and sensitive information has a significant impact on government and business sectors. Every software system creates log files. Hence, the enormous amount of log data each day is exceeding petabytes. The log data from network traffic has high volume, variety, and velocity. Anomalies can be predicted and future attacks can be protected by these log data. The behaviors of hackers can be captured and the possible attacks can be analyzed by log transactions.

An intrusion is an attempt to get illegal access to network resources. Intrusion detection is called monitoring the event signs of unwanted activities. It is a reliable system for network traffic to decide whether there is an attack or not. It becomes the best way for the security problem of the network system compared with other defense systems.

The intrusion detection system can be defined as an attack when a pattern is different from previous normal patterns. Streaming transactions with huge sized input causes a false alarm. Therefore, it produces thousands of alarms per day. However, 99% of these alarms are false alarms. False-positive, wrongly considered an attack, is still the most important problem [6]. Deep learning-based auto-encoder using intrusion detection can decrease false alarm. It can detect not only known intrusions but also works well with unknown intrusions [5].

Big data analytics have been recently developed and widely used in various areas such as banking, insurance, healthcare, education, manufacturing industries and risk and fraud management, etc. Big data analytics aids organizations to profile customers based on different features. Applying analytics to big data can improve performance and extract valuable information. Big data analytics can detect suspicious activities and investigate the historical data to predict future attacks. It can handle unstructured, semi-structured, structured data that are being generated daily [5].

The main motivation of the intrusion detection system is to detect attacks from a huge amount of data with high speed. The heterogeneous data and timeliness processing transactions can be used by big data streaming analytics. Big data streaming analytics using Apache Kafka and Spark Streaming can perform intrusion detection in real-time. When a huge amount of streaming data flows into the system within a second, fault tolerance is an issue. Multiple brokers of Apache Kafka and Resilient Distributed Dataset abstraction technique of Apache Spark Streaming can recover loss after a failure occurs [14].

The major purpose of the paper is to investigate the impact of processing time on the number of stream records and to improve the processing efficiency of Apache Kafka and Spark Streaming.

The paper is organized as follows: Section 2 analyzes related studies. Section 3 describes the method used in this investigation. Section 4 discusses the dataset information, the stream processing architecture with experimental results and the deep learning method as future work. Conclusion and further research are presented in Section 5.

2. Related Work

In 2018, Sheeraz et al., in [11] demonstrated big data streaming analysis of detecting anomalies using the KMeans clustering algorithm. The experiment performed anomalies detection by using Apache Kafka and Apache Spark on KDDCUP99 dataset. This study got 76% and 68% average accuracy on single and distributed environments and detection times are up to 33 seconds. The limitations of the paper are to reduce accuracy and decrease the processing time.

In [9], Apache Spark and Spark MLlib (Machine learning Libraries), four machine learning algorithms, namely Native Bayes, Decision Tree, Support Vector Machine, and Random Forest were used to detect anomalies. Support Vector Machine got the highest accuracy rate that is 92%. In this paper, incoming network traffic detection time did not meet in real-time. This is an important issue for further research.

Ichinose et al., in [1] introduced the video analysis framework from various cameras by using Apache Kafka and Spark Streaming on MINIST image dataset. This paper considered Kafka parameter settings to be efficient in data transfer performance. The experiment showed that images per second depend on the number of brokers and the number of partitions in Apache Kafka.

Masabo and Kaawase in [5] compared malware detection by using machine learning SVM (Support Vector Machine) and deep learning based on big data analytics. In this study, deep learning got 97% accuracy and Support Vector Machine got 95% accuracy. The future work of this paper is going to increase accuracy and processing speed.

Jayanthi and Sumathi published a paper that compared the difference between Hadoop MapReduce and Apache Spark in the weather forecast analysis in [3]. This system reported that Apache Spark succeeded in dealing with the drawback of computational processing speed in Hadoop. In this paper, Apache Spark is 1.5x better performance than the MapReduce.

Sathyapriya et al., in [8] reported the comprehensive review of big data frameworks such as Hadoop, MapReduce, Spark and Flink in credit card fraud detection. According to the results of this paper, it suggested that Apache Spark is more efficient than others.

In [10], Noac'H et al., attempts to show that the different parameter configurations have an impact on the performance metrics of Apache Kafka. The major

objective of this study was to help stream processing developers to avoid the bottleneck.

3. Background Theory

In this section, Apache Kafka and Spark Streaming are highlighted as a stream processing architecture because the integration of them can support real-time big data analytics. Streaming processing architecture consists of 2 layers. The first layer is Apache Kafka, which ingests the streaming data and the second layer is Apache Spark Streaming which processes generated data streams.

3.1. Data Ingestion by Apache Kafka

Kafka is the most popular distributed publish-subscribe messaging system. It consists of topics, brokers, producers and consumers. Kafka organizes a stream of messages called topics. Producer distributes streaming messages, and consumer retrieves these messages [10].

A Kafka comprises one or more servers called brokers that collect and provide reliable data storage and then publish related topics. Apache Zookeeper is used to track the status of cluster nodes in Kafka [1]. Producers send messages to a broker. The broker keeps this message and the consumer collects this data without loss. There is a broker in a single node cluster as shown in Figure 1.

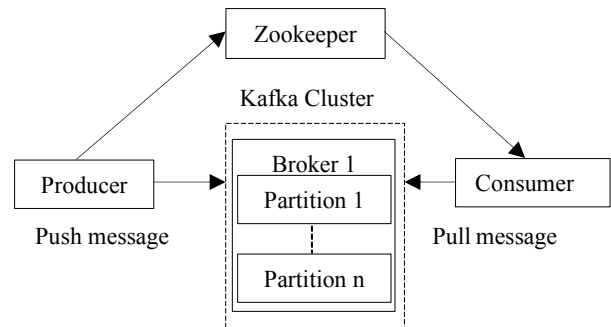


Figure 1: Configuration of Apache Kafka architecture using a single broker

If a broker is down, the stream data processing will stop and all of the streaming transactions will be lost. Nevertheless, a broker fails in a multi-broker system which is normally working without being affected. Hence, Kafka's multi-broker in Figure 2 has the advantages of fault-tolerance.

Every broker has several partitions [1]. A partition acts as a leader and others act as followers. The leader takes to read and write operations and followers replicate the leader. When the leader fails, followers automatically select the new leader. Therefore, multiple partitions in the Kafka process in parallel and then get better processing time.

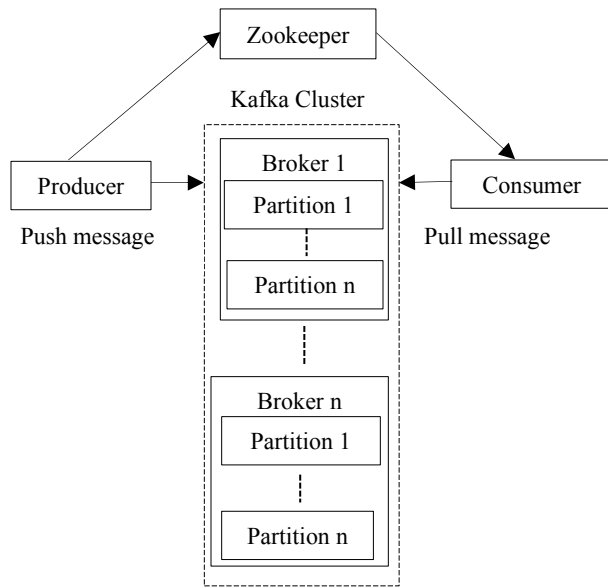


Figure 2: Configuration of Apache Kafka architecture using multi-brokers

3.2. Data processing by Apache Spark Streaming

Streaming data in online transactions may be considered as continuous streaming transactions happening rapidly. The vast amount of streaming transactions flows at fast speed over the network. Streaming analytics rapidly processes when streaming transactions arrive. Streaming processing based on a big data framework can deal with high-speed real-time data. The number of real-time applications of streaming data has been increased because of the Internet of Thing (IoT), mobile devices, social networks, and sensors, etc. Streaming analytics are found across industries such as credit card fraud detection, intrusion detection and data generated by the Internet of Things (IoT) sensors, etc. Therefore, streaming analysis is more and more important to provide high performance and accurate information as well as to handle simultaneously increasing data.

Apache Spark is an open-source, distributed, fast and cluster computing framework for an enormous amount of data. Apache Spark is up to 100 times faster than Hadoop because of an in-memory processing engine for large scale parallel processing. It supports various programming languages such as Java, Python, R and Scala. It reduces the time-consuming of reading and writing and processes data in parallel. The libraries of Apache Spark are Spark Core for parallel and distributed processing, Spark Streaming for stream processing, Spark MLlib for machine learning, Spark SQL and GraphX for graphic processing [3].

Apache Spark Streaming is one of the components of Apache Spark and used for micro-batch streaming

processing. Spark streaming uses in-memory computation and performs stream processing. Spark streaming recovers in node failure without losses than other stream processing frameworks such as Storm, Flink and MapReduce. It does not worry about duplication because it offers exactly-one delivery. The main advantages of Spark Streaming are easy to use, high throughput, load balancing, low latency, highly scalable and fault-tolerant.

Spark Streaming can easily process live data streams and provide results in near real-time. Data can be captured from several sources such as Apache Kafka, Apache Flume, Amazon Kinesis, etc. Finally, processed data streams can be stored in databases and displayed by live dashboards [13].

Apache Spark Streaming receives streaming transactions and splits data into batches as the specified batch interval. And then each batch is processed via a Spark engine that generates the final stream of transactions for each batch interval [11]. It is as shown in Figure 3.

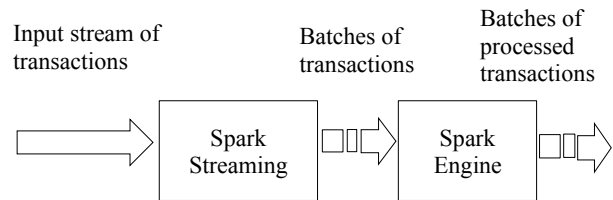


Figure 3. Apache Spark Streaming processing of transactions data

Apache Spark reads data from a file and converts data to a Resilient Distributed Dataset (RDD). RDD helps to perform in-memory computation processes and overcome failure. The continuous stream of data that comes from Spark Streaming is called Discretized Stream (DStream) [13]. The DStream supports the parallel recovery of loss state that advances the efficiency of streaming data. Each RDD in a DStream splits input data into batches according to time interval which is called windowing. The process of Apache Spark Streaming is shown in Figure 4.

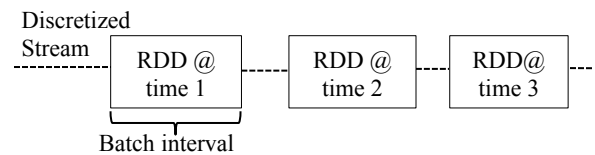


Figure 4. Streaming data processing in Apache Spark Streaming

4. The Stream Processing Architecture

In the proposed intrusion detection framework, Apache Kafka is combined with Apache Spark Streaming for real-time streaming analytics. The steps of the stream

processing architecture are shown in Figure 5. Firstly, network traffic transactions are retrieved from CSV files. To get data streams, these transactions are transmitted via Apache Kafka. Kafka is used ingest network traffic transactions in real-time. It captures streaming data event and then distributes a huge amount of data streams without the loss due to partitioning and replication. After that, Spark Streaming is used to process these stream data. Spark Streaming divides data according to specified the duration of the batch interval. And then the streaming output data for each batch is generated in Spark UI. Finally, the results of performance in Spark UI are evaluated and analyzed. The streaming processing architecture is as shown in Figure 5.

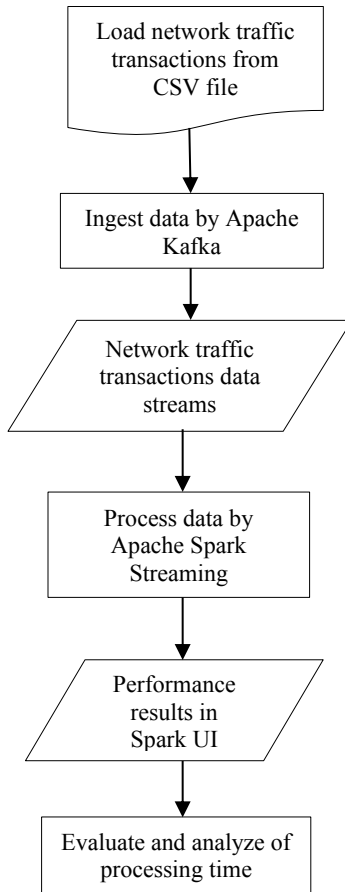


Figure 5. Overview diagram for stream processing architecture

4.1 Dataset

The dataset used for this study is the UNSW-NB15 available at <https://www.unsw.adfa.edu.au/unsw-canberra-cyber/cybersecurity/ADFA-NB15-Datasets>. The network traffic transactions are retrieved from the Cyber Range lab of the Australian Center of Cyber Security for

research purpose. It provides 100 GB of network traffic with pcap files. It also supports CSV files. It consists of 49 attributes with a class label and nine attacks, namely Fuzzers, Backdoors, DoS, Reconnaissance, Exploits, Generic, Analysis, Shellcode and Worms [12].

4.2. Experimental Results

The proposed work is performed by Core i5 processor and 8GB RAM in Ubuntu 18.04 LTS platform. To implement the system, JDK 11, Scala 2.11.12, Apache Kafka 2.12 and Apache Spark 2.1.0 are used to create streaming processing. The enormous growing amount of stream transactions is a challenge for intrusion detection. This system aims to get faster processing time and fault-tolerance in intrusion detection.

At first, the multiple data streams are sent via Apache Kafka and then processed by Spark Streaming. Finally, the performance of Apache Kafka and Spark Streaming is investigated. Different durations of the batch interval of Spark Streaming and the number of partitions in Kafka are evaluated using the UNSW-NB15 dataset.

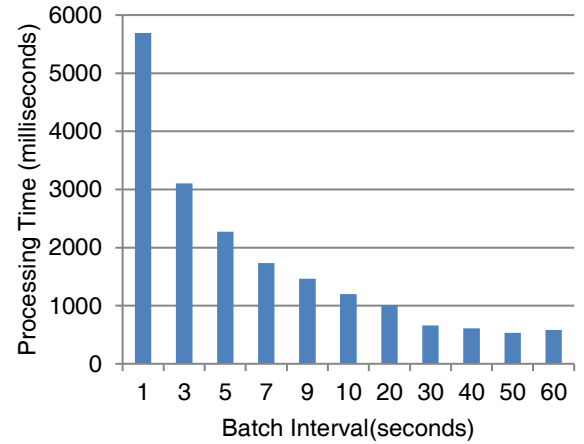


Figure 6. Performance analysis of 500,000 records under different batch intervals

The time-consuming using different streaming batch intervals are shown in Figure 6. The experimental results are evaluated by 500,000 records and the file size is 120 MB. The experimental result shows that the duration of the batch interval has a significant impact on the processing rate of Spark Streaming. If the duration of the batch interval is greater, the time-consuming will be slightly decreased. Consequently, a higher duration of time interval causes rapidly processing speed. However, the scheduling delay time of 60 seconds batch interval takes up to 10 milliseconds for each batch. And then, it does not support real-time decisions. According to this experiment, the batch interval should be defined between 10 seconds and 50 seconds in UNSW-NB 15 dataset. Hence, the duration of the 10 seconds, 30 seconds and 50

seconds batch intervals will be evaluated in other experiments.

Figure 7 compares the processing time according to different records of the 10 seconds, 30 seconds and 50 seconds batch intervals. The experimental results are analyzed in 100000, 300000 and 500000 records and the file sizes of this record are 23.8MB, 71.2MB and 120MB respectively. According to the results, the processing speed is slightly increased when the numbers of records are increased. The 50 seconds batch interval has less execution time than 10 seconds batch interval as shown in Figure 7.

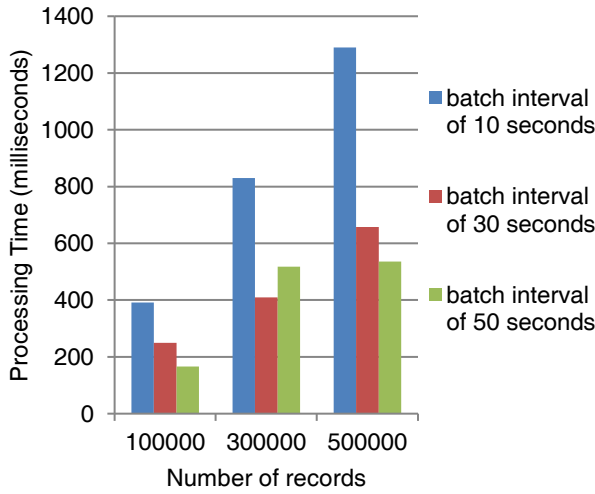


Figure 7. Comparative study of processing time using Kafka and Spark Streaming on different records

Moreover, there are no failure records in this experiment as a result of Apache Spark Streaming of checkpoint mechanism which handles faults. When a failure occurs in Spark Streaming, it can be recomputed from replicating data. Therefore, the integration of Kafka and Spark Streaming provides fault-tolerance and an enormous amount of transactions in real-time insight on incoming data.

Figure 8 illustrates the experimental results of time elapse and the number of partitions between 10 seconds, 30 seconds and 50 seconds batch intervals. It can be observed that the processing time is slightly decreased when the number of partitions is increased in three different batches. According to the results, 50 seconds batch interval performs better than 10 seconds batch interval.

In this experiment, the 10 seconds, 30 seconds and 50 seconds batch intervals perform with less scheduling delay. The scheduling delay is 1 or 2 milliseconds for each batch. However, it is to be aware that the higher batch interval may cause a higher scheduling delay.

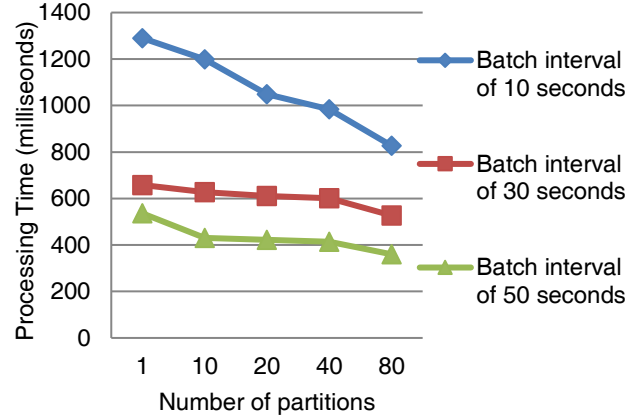


Figure 8. Comparison between the processing time in 500,000 records per partitions of 10 seconds, 30 seconds and 50 seconds batch intervals

4.3. Future Work by Deep Learning-based Auto-encoder

For future work, the auto-encoder is going to deploy network traffic transaction data in a big data framework. The combination of streaming analytics and deep learning has been studied in recent years. A deep learning-based auto-encoder is going to evaluate the accuracy, sensitivity, specificity, and prediction time. Auto-encoder expects to get more improvement in accuracy than Machine Learning libraries in Apache Spark. Now, the results cannot display because this is an ongoing research project.

The auto-encoder is an unsupervised artificial neural network. Therefore, it does not need a huge amount of labeled data. Auto-encoder can extract deep features more effectively from intrusion detection transactions and accurately detect attacks. It learns automatically deep features from unlabeled data and obtains better features and a high level of accuracy. The auto-encoder is used for dimensionality reduction, feature learning, and outlier detections. It is a powerful feature detector because it can extract relevant features. It contains an input layer, a hidden layer, and an output layer [2]. The output value equals the input value in auto-encoder as shown in Figure 9. The learning process discovers the reconstruction error between the original data and output data. The reconstruction error of the auto-encoder is used to define the intrusion score. If the reconstruction error is larger than the normal transaction score, it can be defined as intrusion [7]. The two stages of auto-encoder are encoding and decoding.

An encoder compresses data to delete unnecessary information from the input layer to the hidden layer through the use of equation (1).

$$H = \sigma (W_{ij} X + \varphi_1) \quad (1)$$

A decoder reconstructs the original data from the hidden layer by using equation (2).

$$Y = \sigma (W_{jk} H + \varphi_2) \quad (2)$$

Where, $X = (x_1, x_2, \dots, x_n)$ is the input vector, $Y = (y_1, y_2, \dots, y_n)$ is the reconstruction error and $H = (h_1, h_2, \dots, h_n)$ is the output vector. W_{ij} is the weight connection matrix between the input layer and the hidden layer. W_{jk} is the weight connection matrix between the hidden layer and the output layer. φ_1 and φ_2 are the bias vectors of the input layer and the hidden layer. The sigmoid function σ is an activation function [21].

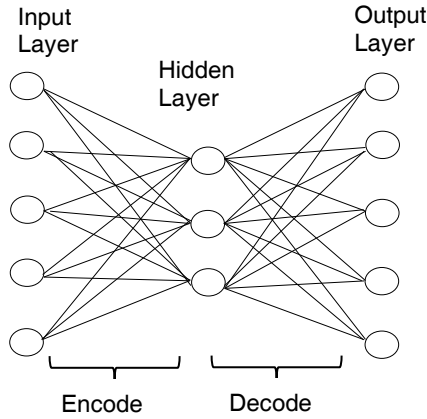


Figure 9. The structure of auto-encoder representation

5. Conclusion and Further Extension

At present, big data analytics is the main key to the security and privacy challenges. The stream processing architecture increases the processing speed of intrusion detection. According to the outcomes, multiple partitions of Apache Kafka and the batch intervals between 10 seconds and 50 seconds of Spark Streaming get better performance in the integration of Kafka and Spark Streaming.

For further development, big data and deep learning are going to apply in real-time network traffic LAN security monitoring system at the University of Information Technology. The integration of big data and deep learning method can be expected to get a higher detection rate, decrease computation complexity, reduce false alarms, and detect intrusion attack patterns.

6. References

- [1] A. Ichinose, A. Takefusa, H. Nakada and M. Oguchi, "A study of a video analysis framework using Kafka and Spark streaming", In 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 2396-2401.
- [2] B. Yan and G. Han, "Effective feature extraction via stacked sparse autoencoder to improve intrusion detection system", IEEE Access, 2018.
- [3] D. Jayanthi and G. Sumathi, "Weather data analysis using spark—an in-memory computing framework", In 2017 Innovations in Power and Advanced Computing Technologies (i-PACT), IEEE, 2017, pp. 1-5.
- [4] D.S. Berman, A.L. Buczak, J.S. Chavis and C.L. Corbett, "A survey of deep learning methods for cyber security", 2019.
- [5] E. Masabo and K.S. Kaawase, "Big data: deep learning for detecting malware", Proceedings of the 2018 International Conference on Software Engineering in Africa, 2018.
- [6] F. Xiao and X. Li, "Using outlier detection to reduce false positives in intrusion detection", In 2008 IFIP International Conference on Network and Parallel Computing, IEEE, 2018, pp. 26-33.
- [7] J. An, and S. Cho, "Variational autoencoder based anomaly detection using reconstruction probability", 2015.
- [8] M.S. Sathyapriya and D.V. Thiagarasu, "Big Data Analytics Techniques for Credit Card Fraud Detection: A Review", *International Journal of Science and Research (IJSR)*, 2017."
- [9] M. Belouch, S. El Hadaj and M. Idhammad, "Performance evaluation of intrusion detection based on machine learning using Apache Spark", 2018.
- [10] P. Le Noac'H, A. Costan and L. Bougé, "A performance evaluation of Apache Kafka in support of big data streaming applications", In 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 1-5.
- [11] S. N. Lighari, "The Efficient Way of Detecting Anomalies in Large Scale Streaming Data", *University of Sindh Journal of Information and Communication Technology*, 2018.
- [12] <https://www.unsw.adfa.edu.au/unsw-canberra cyber/>.
- [13] <https://spark.apache.org/docs/latest/>.
- [14] <https://data-flair.training/blogs/>
- [15] <https://businesstech.co.za/news/general/185177/>.